

ABM BUSINESS LOGIC BIBLE

SECTION 2 — MASTER ENTITY MODEL

Full 5-Whys Excavation · 33 Entities · 5 Tiers
Decimal Technologies × KSA Banking & FI Ecosystem

Built on the Section 1 foundation (409 rules → 25 meta-principles + ~230 operational rules + 8-step Universal Decision Sequence). Every entity excavated via iterative 5-Whys, surfacing new data-layer rules and a complete field specification for engineering handoff.

Entities	33	5-Whys analyzed	165
New rules surfaced	163	Tiers	5

ENTITY TIER ARCHITECTURE

After excavating 409 rules across Section 1, the system requires 33 entities organized in 5 tiers. The original FSD had 9 flat entities; the rebuilt model treats each entity as a business object the system independently creates, reads, updates, history-tracks, and decides about.

TIER 1 — CORE (5)	Account · Contact · Signal · Message · Engagement_Event
TIER 2 — INTELLIGENCE (10)	Three_Score_Record · Enrichment · Buying_Window · Buying_Committee_Member · Initiative · Vendor_Intelligence · Organizational_Assessment · Relationship_Edge · Connector · Market_Interaction_History
TIER 3 — DECISION (8)	Reasoning_Chain · Outreach_Sequence · Account_Strategy · Portfolio_Allocation · Conflict_Resolution · Decision_Pattern · Risk_Assessment · Human_Review_Record
TIER 4 — GOVERNANCE (6)	Trust_Capital_Ledger · System_Event · Rule_Registry · AI_Model_Registry · Prompt_Version · Capability_Gap
TIER 5 — ANALYTICS (4)	Outcome_Attribution · Timing_Record · Feedback_Loop_Entry · Performance_Baseline

The Test for an Entity: "Does the system need to independently create, read, update, track history, and make decisions about this thing?" If yes → it is an entity.

TIER 1 — CORE ENTITIES

The fundamental objects the entire system revolves around.

ENTITY 1: ACCOUNT

Any organization in the KSA banking/FI ecosystem Decimal could sell to. The relationship container and decision anchor.

WHY #1 — Why does the Account entity need to change from the original flat record?

The original Account had ~15 flat fields. The 409 rules revealed an Account is a living decision system: three independent scores, window state, buying-journey stage, consensus dynamics, champion tracking, budget status, initiative maturity, organizational stability, vendor entrenchment, procurement readiness, and a 10-dimension trajectory. The flat record becomes a rich intelligence object.

WHY #2 — Should ALL of this live on Account, or in separate entities?

Separate. Account stores IDENTITY and CURRENT STATE. Each intelligence dimension has its own lifecycle, update frequency, and history, so each becomes a linked entity. Otherwise history tracking is impossible and the entity becomes a 100-field monster.

WHY #3 — What is the MINIMUM that MUST live on Account itself?

Only what identifies the account, routes it to the right signals/playbooks, and answers 'what is the system's posture right now' without joining ten tables. This justifies a denormalized current_state cache — a deliberate exception to normalization, because the decision engine reads posture thousands of times per cycle.

NEW RULE — ACCT-STATE-001: Account maintains a denormalized current_state block (tier, window_state, readiness, stage, last_decision_ts) updated by the scoring engine on every recalculation. Read-optimized cache; authoritative history lives in the intelligence entities.

WHY #4 — Why does that denormalization not create an integrity risk?

Because the cache is write-controlled: only the scoring engine writes current_state, and every write emits a System_Event. If cache and source disagree, the event log reconstructs truth. The Account never accepts a direct human edit to current_state.

NEW RULE — ACCT-STATE-002: current_state fields are system-writable only. Human changes must target the underlying intelligence entity, which retriggers recalculation. Direct writes to current_state are rejected.

WHY #5 — What is the deepest reason Account exists?

Because the unit of selling is the organization, not the person — Principle 2 made physical. The Account is the relationship container and decision anchor: the object against which Effective Opportunity and Decision Score resolve. Without it the system is a contact spammer, not an ABM engine.

NEW RULE — ACCT-IDENTITY-005: Every signal, contact, score, window, and reasoning chain MUST resolve to exactly one Account. Orphaned intelligence is quarantined in Unresolved_Intelligence — never silently dropped or attached.

ACCOUNT — Field Specification

Field	Type	Business Rule	Source
account_id	UUID	System-generated, immutable	—
account_name	String	Unique; legal or common name	ACCT-IDENTITY-001
aliases	Array[String]	Legal/Arabic/abbrev/prior names for entity resolution	ACCT-IDENTITY-003
segment	Enum	ISLAMIC_BANK / DIGITAL_BANK / COMMERCIAL_BANK / BNPL / EMBEDDED_FINANCE / SME_LENDER / OPEN_BANKING / INFRASTRUCTURE	ACCT-IDENTITY-001
country	String	MVP: 'SA' only	ACCT-IDENTITY-004

Field	Type	Business Rule	Source
relevance_scope	Array[String]	Divisions Decimal is relevant to	ACCT-RELEVANCE-001
current_state	Object	Denormalized cache: tier, window_state, readiness, stage, last_decision_ts	ACCT-STATE-001/002
lifecycle_status	Enum	PROSPECT / ENGAGED / PILOT / CUSTOMER / DORMANT / DISQUALIFIED	ACCT-LIFECYCLE-001
created_at / updated_at	Timestamp	Immutable created; system-managed updated	—

ENTITY 2: CONTACT

A person inside (or formerly inside) an account, with persistent person-level identity across job changes.

WHY #1 — Why is Contact separate from Account?

Organizations don't decide — people do. Account is the container; Contact is a person with their own role, motivations, influence, and trajectory. The same person can move between accounts, so Contact must persist independently.

NEW RULE — CONTACT-IDENTITY-001: A Contact is identified by person-level identity (name + verified email/LinkedIn), NOT by employer. On job change, Contact persists; only current_account_id and role change. Prior history is retained.

WHY #2 — Why person-persistence across job changes?

Relationship capital is personal. A champion who moves from SNB to Al Rajhi carries the Decimal relationship. MARKETMEM-002 ('interacted with this PERSON before at any account?') is impossible if identity resets on job change.

NEW RULE — CONTACT-MEMORY-001: Each Contact carries interaction_lineage — full cross-account engagement/sentiment/outcome history, preserved across employer changes. Feeds Market_Interaction_History.

WHY #3 — What distinguishes Contact from Buying_Committee_Member?

Contact is the raw person record (identity, contactability, persona). Buying_Committee_Member is a role-in-a-deal (influence weight, champion status, consensus position). One Contact can be a committee member on zero or many deals.

NEW RULE — CONTACT-ROLE-001: Contact stores identity and persona; deal-specific positioning lives on Buying_Committee_Member. A Contact with no active deal positioning is still valid.

WHY #4 — Why must persona live on Contact, not be inferred per message?

Persona (motivations, risk posture, decision style) is stable and expensive to derive. Re-inferring per message wastes capital and breeds inconsistency. Storing it makes reasoning Step 4 deterministic and auditable.

NEW RULE — CONTACT-PERSONA-001: Contact stores structured persona_profile (role_motivations, decision_style, risk_posture, communication_preference, evidence_orientation) with provenance + confidence. Stale personas (>180 days or post-role-change) flagged for re-enrichment.

WHY #5 — Deepest reason Contact exists?

Trust is built person-to-person; the system must never treat a human as a disposable address. Contact embodies the Relationship and Market Memory principles — forcing the system to remember who it spoke to and what it owes them.

NEW RULE — CONTACT-GOVERNANCE-001: No outreach may be generated to a Contact without checking interaction_lineage for prior negative sentiment, do-not-contact flags, or active fatigue. A burned relationship is a hard gate, not a soft penalty.

CONTACT — Field Specification

Field	Type	Business Rule	Source
contact_id	UUID	Immutable; persists across jobs	CONTACT-IDENTITY-001
full_name	String	Name + Arabic variant	CONTACT-IDENTITY-001
verified_email / linkedin_url	String	At least one verified identifier required	CONTACT-IDENTITY-001
current_account_id	UUID	FK to Account; updates on job change	CONTACT-IDENTITY-001
current_role / seniority	String/Enum	Title + decision-tier band	CONTACT-ROLE-001
persona_profile	Object	Motivations, decision style, risk posture + confidence	CONTACT-PERSONA-001
interaction_lineage	Array[Object]	Cross-account engagement/sentiment history	CONTACT-MEMORY-001
contact_flags	Object	do_not_contact, fatigue_state, prior_negative_sentiment	CONTACT-GOVERNANCE-001
created_at / updated_at	Timestamp	System-managed	—

ENTITY 3: SIGNAL

The atom of the event-driven architecture: a durable, timestamped piece of evidence about a change in the world.

WHY #1 — Why is Signal an entity, not a transient trigger?

Signals are the evidence base. Every score, decision, and message traces back to signals. Transient signals make reasoning unauditable — you can't answer 'why did we reach out?' months later.

NEW RULE — SIGNAL-PERSISTENCE-001: Every signal is stored immutably with raw source payload, detection timestamp, and source attribution — even after scoring/action. Signals decay in weight, not existence; never deleted.

WHY #2 — Why store the raw payload, not just the parsed signal?

Parsing logic evolves. A signal parsed under v1 may need reinterpretation under v2. Raw payload allows reprocessing and protects against parser bugs corrupting evidence.

NEW RULE — SIGNAL-PROVENANCE-001: Signal stores raw_payload (immutable) and parsed_interpretation (versioned, parser_version). Reparsing creates a new interpretation linked to the same raw payload; original retained.

WHY #3 — Why a decay model on the entity itself?

Timing is the primary competitive weapon. A CTO-hiring signal is P1-hot at week 1 and noise at month 6. Current weight is a function of age, so the decay curve is a property of signal type, computed at read time.

NEW RULE — SIGNAL-DECAY-001: Each Signal carries a signal_type whose decay function (half-life, hard-expiry) sets current weight = base_priority x decay(age). Expired signals remain stored, contribute zero weight, and cannot independently trigger outreach.

WHY #4 — Why must Signal carry deduplication identity?

The same event arrives via multiple sources (funding round on Crunchbase, LinkedIn, news). Counting it thrice corrupts scoring. The entity needs an event-identity key to collapse duplicates while preserving corroboration.

NEW RULE — SIGNAL-DEDUP-001: Each Signal computes an event_fingerprint (entity + event_type + time-window). Signals sharing a fingerprint collapse into one logical signal with multiple corroborating_sources; corroboration raises confidence but does NOT multiply weight.

WHY #5 — Deepest reason Signal exists?

The entire system is event-driven (Principle 1); Signal is the atom of that architecture. It is trigger, evidence, and audit trail at once. Remove Signal and the engine is blind.

NEW RULE — SIGNAL-PRIMACY-001: No account may enter or advance in outreach without at least one qualifying, non-expired, deduplicated signal. The triggering signal is permanently linked to the resulting Reasoning_Chain.

SIGNAL — Field Specification

Field	Type	Business Rule	Source
signal_id	UUID	Immutable	SIGNAL-PERSISTENCE-001
account_id	UUID	FK to Account (or Unresolved_Intelligence)	ACCT-IDENTITY-005
signal_type	Enum	Drives priority + decay curve	SIGNAL-DECAY-001
priority	Enum	P1 / P2 / P3 base weight	SIGNAL-DECAY-001
raw_payload	Object	Immutable source data	SIGNAL-PROVENANCE-001
parsed_interpretation	Object	Versioned parse (parser_version)	SIGNAL-PROVENANCE-001
event_fingerprint	String	Dedup key	SIGNAL-DEDUP-001
corroborating_sources	Array	Multi-source confirmations to confidence	SIGNAL-DEDUP-001
detected_at / expires_at	Timestamp	Drives decay + hard expiry	SIGNAL-DECAY-001

ENTITY 4: MESSAGE

Every outbound interaction — the point where strategy becomes contact with the real world and brand capital is spent.

WHY #1 — Why is Message a first-class entity, not a log line?

Every outbound interaction is a learning unit and audit artifact — the foundation for AI learning, engagement analysis, sequence optimization, tone evaluation, conversion analysis. A log line can't carry that.

NEW RULE — MESSAGE-RECORD-001: Every outbound message — sent, queued, or QC-rejected — is stored with full content, channel, sequence position, and the Reasoning_Chain that produced it. Rejected drafts retained for failure analysis.

WHY #2 — Why link every Message to its Reasoning_Chain?

Principle 5 forbids generating messages from raw data — every message must have a traceable thought process. Without the link you can't diagnose whether a bad outcome came from bad reasoning or bad execution.

NEW RULE — MESSAGE-TRACE-001: No Message may exist without a reasoning_chain_id. A message with no reasoning lineage is rejected before send.

WHY #3 — Why store prompt_version and AI metadata on Message?

Messages are generated by evolving prompts. To optimize and enable rollback, each message records which prompt version and model produced it — tying outcomes to specific generation logic.

NEW RULE — MESSAGE-AI-PROVENANCE-001: Each AI-generated Message stores prompt_version, model_id, ai_generated, and hallucination_score from QC. Links to Prompt_Version and AI_Model_Registry.

WHY #4 — Why store response sentiment on Message rather than separately?

The message and its reaction are a matched pair — the unit of learning is 'this content to this response.' Splitting them breaks the feedback loop that teaches which tone and angle convert.

NEW RULE — MESSAGE-OUTCOME-001: Each Message records response_received, response_sentiment, response_latency. Feeds Outcome_Attribution and Feedback_Loop_Entry; sentiment also updates Contact.interaction_lineage.

WHY #5 — Deepest reason Message exists?

Message is where strategy meets reality and the system is most accountable — the point of brand-capital expenditure. Storing it fully keeps the system honest and prevents repeating a mistake on the same person.

NEW RULE — MESSAGE-CAPITAL-001: Every sent Message debits the relevant capital ledgers (brand, relationship, connector) and links to the Trust_Capital_Ledger entry it generated, making the cost of each touch explicit.

MESSAGE — Field Specification

Field	Type	Business Rule	Source
message_id	UUID	Immutable	MESSAGE-RECORD-001
account_id / contact_id	UUID	FKs	—
reasoning_chain_id	UUID	Mandatory link	MESSAGE-TRACE-001
sequence_id / sequence_step	UUID/Int	Workflow position	—
channel	Enum	EMAIL / LINKEDIN	—
content	Text	Full message body	MESSAGE-RECORD-001
status	Enum	DRAFT / QC_REJECTED / QUEUED / SENT	MESSAGE-RECORD-001
ai_generated / prompt_version / model_id / hallucination_score	Mixed	AI provenance	MESSAGE-AI-PROVENANCE-001
response_received / response_sentiment / response_latency	Mixed	Outcome capture	MESSAGE-OUTCOME-001
capital_ledger_id	UUID	Link to cost incurred	MESSAGE-CAPITAL-001

ENTITY 5: ENGAGEMENT_EVENT

Recipient-side actions (open, click, reply, profile view, meeting accept) — the system's sensory return path from reality.

WHY #1 — Why separate Engagement_Event from Message?

A Message is something we send; an Engagement_Event is something they do. Different actors, different timelines. Conflating them loses the asymmetry between our action and their reaction.

NEW RULE — ENGAGE-SOURCE-001: Engagement_Event records recipient-side or third-party-observed actions, distinct from outbound Messages. Each records actor, action_type, and source-of-observation.

WHY #2 — Why feed scoring in near-real-time?

Engagement is a signal — an inbound buying-intent indicator. A reply or repeated profile views should retrigger scoring (Principle 1).

NEW RULE — ENGAGE-TRIGGER-001: High-intent Engagement_Events (reply, meeting-accept, repeated content interaction) retrigger account scoring and may open/widen a Buying_Window. Engagement is a first-class signal class.

WHY #3 — Why discrete events rather than aggregate counts?

Sequence and timing carry meaning (three opens in an hour != three opens over three weeks). Aggregates destroy the temporal pattern distinguishing intent from noise.

NEW RULE — ENGAGE-GRANULARITY-001: Engagement is stored as discrete timestamped events, never pre-aggregated. Aggregations computed at read time so velocity, clustering, and recency remain recoverable.

WHY #4 — Why link to the specific Message that provoked it?

Attribution requires the cause-effect pair. To learn which message earned the reply, the event must point back to the triggering Message and its Reasoning_Chain.

NEW RULE — ENGAGE-ATTRIBUTION-001: Where traceable, an Engagement_Event stores triggering_message_id, enabling Outcome_Attribution to credit the specific reasoning and prompt version.

WHY #5 — Deepest reason Engagement_Event exists?

It is the system's only feedback from reality about whether its strategy worked. It makes the system a learning loop rather than a one-way broadcaster — proof this is intelligence, not automation.

NEW RULE — ENGAGE-LEARNING-001: Every Engagement_Event contributes to a Feedback_Loop_Entry. Absence of engagement after N touches is itself a learning signal (triggers sequence pause / strategy review).

ENGAGEMENT_EVENT — Field Specification

Field	Type	Business Rule	Source
event_id	UUID	Immutable	ENGAGE-SOURCE-001
account_id / contact_id	UUID	FKs	—
action_type	Enum	OPEN / CLICK / REPLY / PROFILE_VIEW / CONTENT_INTERACTION / MEETING_ACCEPT	ENGAGE-SOURCE-001
triggering_message_id	UUID	FK to Message (if traceable)	ENGAGE-ATTRIBUTION-001
observed_via	Enum	Source of observation	ENGAGE-SOURCE-001
intent_class	Enum	HIGH / MEDIUM / LOW (drives retrigger)	ENGAGE-TRIGGER-001
occurred_at	Timestamp	Discrete, never aggregated	ENGAGE-GRANULARITY-001

TIER 2 — INTELLIGENCE ENTITIES

The objects that store what the system KNOWS about the world.

ENTITY 6: THREE_SCORE_RECORD

A point-in-time snapshot of an account's three independent scores (Fit/ICS, Intent/Dynamic, Intelligence Confidence) with full dimension breakdown.

WHY #1 — Why is scoring a separate entity, not fields on Account?

Scores change constantly and must be history-tracked for explainability and recalibration. Storing only the latest score on Account destroys the trail needed to answer 'why was this HOT last month?' and to retrain weights.

NEW RULE — SCORE-RECORD-001: Each scoring run writes an immutable Three_Score_Record snapshot (fit/ICS, intent/dynamic, intelligence-confidence) with per-dimension contributions and the weight-set version used. The latest snapshot updates Account.current_state.

WHY #2 — Why THREE scores instead of one?

Fit (does this account match our ICP?), Intent (are they showing buying behavior now?), and Intelligence Confidence (how sure are we, given data quality?) are orthogonal. Collapsing them hides why an account ranks where it does and lets low-confidence data masquerade as high intent.

NEW RULE — SCORE-TRIAD-001: Scores are stored as three independent dimensions and never collapsed into a single number at storage time. The Effective Opportunity equation consumes them separately; ICS gates intent so low-confidence cannot inflate priority.

WHY #3 — Why store per-dimension contributions, not just totals?

Black-box scoring has no explainability (a named pitfall). To justify outreach to a human reviewer and to debug mis-scoring, each contributing dimension's value and weight must be recoverable.

NEW RULE — SCORE-EXPLAIN-001: Every Three_Score_Record stores dimension-level breakdown: each input, its raw value, applied weight, and contribution. A score with no breakdown is invalid.

WHY #4 — Why version the weight-set on each record?

Weights are recalibrated monthly using engagement data. Comparing scores across time is meaningless unless you know which weight-set produced each. Versioning enables apples-to-apples trend analysis and safe rollback.

NEW RULE — SCORE-VERSION-001: Each record references a weight_set_version from Rule_Registry. Recalibration creates a new version; historical records keep their original version for valid longitudinal comparison.

WHY #5 — Deepest reason this entity exists?

It is the quantified judgment of the system — the bridge between raw intelligence and prioritized action. It exists so that prioritization is explainable, auditable, and improvable rather than a magic number.

NEW RULE — SCORE-PRIMACY-001: No prioritization, sequencing, or capital allocation may reference a score not backed by a stored Three_Score_Record. Ranking on ephemeral or unstored scores is prohibited.

THREE_SCORE_RECORD — Field Specification

Field	Type	Business Rule	Source
score_record_id	UUID	Immutable snapshot	SCORE-RECORD-001
account_id	UUID	FK to Account	—
fit_score_ics	Int 0-100	ICP match / Ideal Customer Score	SCORE-TRIAD-001
intent_score_dynamic	Int 0-100	Current buying-behavior score	SCORE-TRIAD-001
intelligence_confidence	Int 0-100	Data-quality confidence; gates intent	SCORE-TRIAD-001
dimension_breakdown	Array[Object]	Per-input value, weight, contribution	SCORE-EXPLAIN-001
weight_set_version	String	FK to Rule_Registry weight version	SCORE-VERSION-001
computed_at	Timestamp	Snapshot time	SCORE-RECORD-001

ENTITY 7: ENRICHMENT

Deep intelligence profile data fetched from external sources (Apollo, Clearbit, Clay, LinkedIn) about an account or its people.

WHY #1 — Why is Enrichment a separate entity from the facts it produces?

Enrichment is a process record with provenance, cost, and freshness — distinct from the clean facts that land on Account/Contact. Separating the fetch record from the derived fact lets us trace where a fact came from and when it must be refreshed.

NEW RULE — ENRICH-RECORD-001: Each enrichment run is stored with source, query, cost, raw response, and timestamp. Derived facts written to Account/Contact carry a pointer back to the Enrichment record that produced them.

WHY #2 — Why track source and cost per enrichment?

Enrichment consumes system capital (API spend, rate limits). Tracking cost per source enables EROA accounting and prevents over-enriching low-value accounts.

NEW RULE — ENRICH-COST-001: Enrichment records source_cost and counts against the account's capital budget. Enrichment of accounts below a fit threshold requires justification; speculative enrichment of COLD accounts is rate-limited.

WHY #3 — Why store freshness/expiry on enrichment?

Stale enrichment is a named failure mode. Firmographics, headcount, and roles change; acting on year-old data damages credibility. Each fact needs a confidence-decay and re-fetch trigger.

NEW RULE — ENRICH-FRESHNESS-001: Each enriched fact carries source_date and a type-specific staleness window. Stale facts are flagged and must be re-verified before use in reasoning; staleness lowers Intelligence Confidence.

WHY #4 — Why reconcile conflicting sources rather than overwrite?

Apollo and Clearbit may disagree. Last-write-wins silently destroys signal. The system must record both and resolve by source-reliability, preserving the conflict for audit.

NEW RULE — ENRICH-RECONCILE-001: When sources conflict, both values are retained with source-reliability weighting; the resolved value records its rationale. Unresolved high-stakes conflicts lower confidence and may trigger human review.

WHY #5 — Deepest reason Enrichment exists?

It is how the system converts the unknown into the known — the sensory apparatus that turns a bare account name into an actionable intelligence profile. Without provenance and freshness, knowledge silently rots.

NEW RULE — ENRICH-PRIMACY-001: No persona inference, pain hypothesis, or buying-committee map may be built on facts lacking an Enrichment provenance link. Unsourced 'facts' are treated as hypotheses, not evidence.

ENRICHMENT — Field Specification

Field	Type	Business Rule	Source
enrichment_id	UUID	Immutable	ENRICH-RECORD-001
account_id / contact_id	UUID	Target of enrichment	—
source	Enum	APOLLO / CLEARBIT / CLAY / LINKEDIN / OTHER	ENRICH-RECORD-001
query / raw_response	Object	Provenance of derived facts	ENRICH-RECORD-001
source_cost	Number	Capital accounting	ENRICH-COST-001
source_date / staleness_window	Date	Freshness + re-fetch trigger	ENRICH-FRESHNESS-001
conflict_resolution	Object	Retained conflicts + rationale	ENRICH-RECONCILE-001
fetched_at	Timestamp	Run time	ENRICH-RECORD-001

ENTITY 8: BUYING_WINDOW

A time-bounded period of elevated buying readiness for an account, with open/peak/closing state — timing made into a tracked object.

WHY #1 — Why is the buying window an entity, not a flag?

Timing is the primary competitive weapon (Principle 4). A window has a lifecycle (opens, peaks, decays, closes) and a cause. A boolean flag can't carry the state machine or the multiplier the Effective Opportunity equation needs.

NEW RULE — WINDOW-RECORD-001: A Buying_Window is created when window-opening signals fire (new license, exec hire, funding, mandate). It records open_state (OPENING/PEAK/CLOSING/CLOSED), trigger signals, and a Window Multiplier consumed by the opportunity equation.

WHY #2 — Why track window state transitions rather than just 'open/closed'?

The optimal moment is near peak, not at first detection (acting too early wastes capital; too late misses it). The state machine encodes WHEN within the window to act.

NEW RULE — WINDOW-STATE-001: Window state transitions are timestamped and event-driven. The Window Multiplier is highest at PEAK and decays toward CLOSING. Outreach timing decisions read window state, not merely window existence.

WHY #3 — Why attach the causing signals to the window?

A window without a known cause can't be reasoned about or defended. Linking the trigger signals lets the message reference the real catalyst and lets the system learn which signal types open real windows.

NEW RULE — WINDOW-CAUSE-001: Each Buying_Window links the signals that opened it. Windows with no qualifying signal cause are invalid. Closed-window signals are retained to train window-prediction models.

WHY #4 — Why predict window closure rather than only observe it?

Reacting after closure is too late. The system must forecast decay to schedule action before the window shuts — this is the difference between timing as weapon and timing as luck.

NEW RULE — WINDOW-PREDICT-001: Each window carries an estimated_closure with confidence, updated as signals age. Approaching closure raises execution urgency in the decision sequence and may pre-empt lower-priority work.

WHY #5 — Deepest reason Buying_Window exists?

It is the temporal heart of the system — the object that turns 'who to contact' into 'who to contact NOW.' It exists because in enterprise sales, being right at the wrong time is being wrong.

NEW RULE — WINDOW-PRIMACY-001: Timing decisions in the Universal Decision Sequence (step 5) read Buying_Window state. No 'now vs later' decision may be made without consulting the account's active window, if any.

BUYING_WINDOW — Field Specification

Field	Type	Business Rule	Source
window_id	UUID	Immutable	WINDOW-RECORD-001
account_id	UUID	FK to Account	—
open_state	Enum	OPENING / PEAK / CLOSING / CLOSED	WINDOW-STATE-001
window_multiplier	Float	Consumed by Effective Opportunity eq.	WINDOW-RECORD-001
trigger_signal_ids	Array[UUID]	Causing signals	WINDOW-CAUSE-001
opened_at / estimated_closure	Timestamp	Lifecycle + forecast	WINDOW-PREDICT-001

Field	Type	Business Rule	Source
closure_confidence	Int 0-100	Forecast confidence	WINDOW-PRED ICT-001

ENTITY 9: BUYING_COMMITTEE_MEMBER

A Contact positioned within a specific account's deal: their influence, stance, champion/anti-champion status, and consensus position.

WHY #1 — Why separate this from Contact?

Contact is stable person identity; committee membership is fluid deal-positioning. The same person can hold different influence in different deals and change stance over time. Separation keeps identity stable while positioning evolves.

NEW RULE — BCM-RECORD-001: A Buying_Committee_Member links one Contact to one Account-deal context with influence_weight, stance, and role-in-decision. A Contact may have zero or many committee memberships.

WHY #2 — Why track influence weight explicitly?

Consensus scoring (CONSENSUS-002) weights stakeholder positions by influence. Treating all members equally mis-models how enterprise decisions actually form (a skeptical CFO outweighs an enthusiastic analyst).

NEW RULE — BCM-INFLUENCE-001: Each member carries an influence_weight derived from role, budget authority, and political power mapping. Consensus and opportunity-death models consume these weights, not headcounts.

WHY #3 — Why track stance AND its trajectory?

A champion warming is different from a champion cooling even at the same current stance. Momentum predicts deal health; a single snapshot misses the direction of travel.

NEW RULE — BCM-STANCE-001: Each member records stance (CHAMPION/SUPPORTER/NEUTRAL/SKEPTIC/ANTI-CHAMPION) plus stance_trajectory. Anti-champion detection and consensus-momentum models read trajectory, not just current stance.

WHY #4 — Why model the committee as a graph, not a list?

Influence flows between people (a champion who reports to an anti-champion is constrained). Relationships between members determine whether consensus can actually form.

NEW RULE — BCM-NETWORK-001: Committee members link to Relationship_Edge entries capturing reporting lines and influence flows. Consensus feasibility is assessed over the network, not the member list in isolation.

WHY #5 — Deepest reason this entity exists?

Enterprise deals are won or lost in the committee, not the inbox. This entity makes the invisible politics of the buying group explicit, trackable, and reasoned-about — the core of Account-Centric Design.

NEW RULE — BCM-PRIMACY-001: No account strategy may be finalized without a buying-committee map identifying at minimum the economic buyer, technical evaluator, and any anti-champion (or explicitly recording them as unknown, lowering confidence).

BUYING_COMMITTEE_MEMBER — Field Specification

Field	Type	Business Rule	Source
bcm_id	UUID	Immutable	BCM-RECORD-001
contact_id / account_id	UUID	Person + deal context	BCM-RECORD-001
role_in_decision	Enum	ECONOMIC_BUYER / TECHNICAL_EVALUATOR / CHAMPION / USER / BLOCKER	BCM-RECORD-001
influence_weight	Float	Role + budget + political power	BCM-INFLUENCE-001

Field	Type	Business Rule	Source
stance / stance_trajectory	Enum / Trend	Current + direction of travel	BCM-STANCE-001
relationship_edge_ids	Array[UUID]	Links into committee graph	BCM-NETWORK-001
updated_at	Timestamp	System-managed	—

ENTITY 10: INITIATIVE

A known strategic project inside an account (digital onboarding, NEO expansion, open banking) that Decimal could attach to.

WHY #1 — Why track initiatives as entities?

Decimal sells into initiatives, not abstract accounts. A live SME-lending program is the actual buying context. Without modeling initiatives, the system can't connect its value proposition to a funded, sponsored project.

NEW RULE — INIT-RECORD-001: Each known Initiative is stored with type, sponsor, maturity stage, and criticality. Outreach reasoning must map Decimal's value to a specific initiative where one exists.

WHY #2 — Why track initiative maturity stage?

A just-announced initiative buys differently from one mid-build. Maturity determines whether Decimal is early (shaping) or late (displacing an incumbent), which changes the entire strategy.

NEW RULE — INIT-MATURITY-001: Initiative maturity (ANNOUNCED/PLANNING/BUILDING/LIVE/SCALING) is tracked. Project-maturity models set the engagement angle; entering at BUILDING vs LIVE triggers different playbooks.

WHY #3 — Why track initiative criticality?

Not all initiatives are equal. A CEO-sponsored transformation outranks a departmental experiment. Criticality drives how much capital the opportunity justifies.

NEW RULE — INIT-CRITICALITY-001: Each Initiative carries a criticality rating (strategic weight, executive sponsorship, budget size). Criticality feeds the opportunity equation; low-criticality initiatives cap allocated capital.

WHY #4 — Why link initiatives to sponsors and windows?

An initiative without a sponsor is a rumor; an initiative whose sponsor just left is dying. Linking sponsor (a committee member) and any buying window keeps initiative intelligence live, not static.

NEW RULE — INIT-SPONSOR-001: Each Initiative links its executive sponsor (Buying_Committee_Member) and any associated Buying_Window. Sponsor departure triggers re-assessment of initiative viability.

WHY #5 — Deepest reason Initiative exists?

It is the bridge between Decimal's capability and the account's actual spending. It exists because money flows to initiatives — selling to an account without an initiative is selling to no one.

NEW RULE — INIT-PRIMACY-001: Where no initiative can be identified, the account is treated as lower-readiness and outreach shifts to discovery/education rather than solution-selling.

INITIATIVE — Field Specification

Field	Type	Business Rule	Source
initiative_id	UUID	Immutable	INIT-RECORD-001
account_id	UUID	FK to Account	—
initiative_type	Enum	DIGITAL_ONBOARDING / SME_LENDING / NEO_EXPANSION / OPEN_BANKING / POS_FINANCE / OTHER	INIT-RECORD-001
maturity_stage	Enum	ANNOUNCED / PLANNING / BUILDING / LIVE / SCALING	INIT-MATURITY-001

Field	Type	Business Rule	Source
criticality	Object	Strategic weight, sponsorship, budget size	INIT-CRITICALITY-001
sponsor_bcm_id	UUID	Executive sponsor	INIT-SPONSOR-001
buying_window_id	UUID	Associated window (if any)	INIT-SPONSOR-001

ENTITY 11: VENDOR_INTELLIGENCE

What the system knows about incumbent vendors and competitors inside an account: entrenchment, contracts, satisfaction, lock-in.

WHY #1 — Why model competitor/incumbent presence as an entity?

Decimal rarely enters empty space. An incumbent (Backbase, an in-house build, another vendor) shapes whether the play is displacement, coexistence, or wait. Ignoring incumbency produces naive strategies.

NEW RULE — VENDOR-RECORD-001: Each known incumbent/competitor in an account is stored with product footprint, contract status, and entrenchment level. Strategy must account for incumbency where present.

WHY #2 — Why track entrenchment level specifically?

The Entrenchment Modifier in the opportunity equation depends on it. Deep lock-in (integrated, multi-year contract) makes near-term displacement unrealistic and should suppress opportunity scoring.

NEW RULE — VENDOR-ENTRENCH-001: Entrenchment (integration depth, contract length remaining, switching cost) feeds the Entrenchment Modifier. High entrenchment suppresses opportunity unless a renewal window or dissatisfaction signal is present.

WHY #3 — Why track incumbent satisfaction/dissatisfaction?

Entrenchment plus dissatisfaction is the prime displacement opening. A renewal approaching with complaints is a window; the system must capture satisfaction signals to find these.

NEW RULE — VENDOR-SATISFACTION-001: Dissatisfaction signals (complaints, failed projects, public RFPs) are tracked against incumbents. Dissatisfaction + approaching renewal opens a displacement Buying_Window.

WHY #4 — Why track contract/renewal timing?

Displacement is only realistic near renewal. Knowing renewal timing converts a hopeless 'rip and replace' into a well-timed competitive entry.

NEW RULE — VENDOR-RENEWAL-001: Known contract/renewal dates are stored and feed window prediction. Outreach ramps as renewal approaches; mid-contract displacement attempts are de-prioritized absent strong dissatisfaction.

WHY #5 — Deepest reason this entity exists?

It is the system's competitive awareness — the difference between selling into reality and selling into a fantasy of empty space. It exists because every enterprise account already has incumbents.

NEW RULE — VENDOR-PRIMACY-001: Account strategy must state the incumbency posture (greenfield / coexist / displace / wait) backed by Vendor_Intelligence. An unstated competitive posture lowers strategy confidence.

VENDOR_INTELLIGENCE — Field Specification

Field	Type	Business Rule	Source
vendor_intel_id	UUID	Immutable	VENDOR-RECORD-001
account_id	UUID	FK to Account	—
incumbent_name / product_footprint	String/Object	What they run today	VENDOR-RECORD-001

Field	Type	Business Rule	Source
entrenchment_level	Object	Integration depth, switching cost	VENDOR-ENTR ENCH-001
satisfaction_signals	Array	Dissatisfaction / complaints / RFPs	VENDOR-SATIS FACTION-001
contract_renewal_date	Date	Feeds window prediction	VENDOR-RENE WAL-001
competitive_posture	Enum	GREENFIELD / COEXIST / DISPLACE / WAIT	VENDOR-PRIM ACY-001

ENTITY 12: ORGANIZATIONAL_ASSESSMENT

A standing read on the account's organizational health: leadership stability, restructuring, hiring posture, decision-making climate.

WHY #1 — Why assess organizational state as its own entity?

Org stability gates whether a deal can progress at all. A bank mid-reorg or with a departing CTO can't buy, regardless of fit and intent. This is a distinct intelligence dimension from scoring.

NEW RULE — ORG-RECORD-001: Each account carries an `Organizational_Assessment` covering `leadership_stability`, `restructuring_state`, and `decision_climate`. Instability suppresses Strategic Readiness regardless of fit/intent.

WHY #2 — Why track leadership stability specifically?

Champions and sponsors are people; leadership churn vaporizes deals. A departing sponsor is opportunity-death risk; an incoming leader is a fresh window. Stability must be tracked to anticipate both.

NEW RULE — ORG-LEADERSHIP-001: Leadership changes among decision-makers are tracked. Sponsor/champion departure triggers opportunity-death assessment; new-leader arrival may open a vendor-evaluation window.

WHY #3 — Why track restructuring/reorg state?

During reorgs, budgets freeze and decisions stall. Pushing hard during a reorg wastes capital and risks the relationship. The system must detect reorg states to exercise strategic patience.

NEW RULE — ORG-RESTRUCTURE-001: Restructuring/M&A/budget-freeze states are tracked. During such states, outreach shifts to relationship-maintenance, not solution-pushing; aggressive sequences pause.

WHY #4 — Why feed this into readiness rather than the opportunity score?

Org health is about whether WE should engage now, not how attractive the prize is. It belongs to Strategic Readiness/patience, gating engagement, not to raw opportunity attractiveness.

NEW RULE — ORG-READINESS-001: `Organizational_Assessment` feeds the Strategic Readiness gate. Severe instability can hold engagement even for high-opportunity accounts (the Patience Test).

WHY #5 — Deepest reason this entity exists?

It encodes the wisdom that the best prospect at the wrong organizational moment is not a prospect yet. It exists to stop the system from burning capital against organizations that cannot currently act.

NEW RULE — ORG-PRIMACY-001: No engagement decision may ignore a recent adverse `Organizational_Assessment`. Engaging through known severe instability requires explicit human override.

ORGANIZATIONAL_ASSESSMENT — Field Specification

Field	Type	Business Rule	Source
org_assessment_id	UUID	Immutable snapshot	ORG-RECORD-001
account_id	UUID	FK to Account	—
leadership_stability	Object	Decision-maker churn tracking	ORG-LEADERSHIP-001

Field	Type	Business Rule	Source
restructuring_state	Enum	STABLE / REORG / M&A / BUDGET_FREEZE	ORG-RESTRUCTURE-001
decision_climate	Object	Speed/appetite for new vendors	ORG-RECORD-001
readiness_impact	Int	Contribution to readiness gate	ORG-READINESS-001
assessed_at	Timestamp	Snapshot time	ORG-RECORD-001

ENTITY 13: RELATIONSHIP_EDGE

A single directed link in the relationship graph: who knows/reports to/influences/can-introduce whom, with strength and direction.

WHY #1 — Why model relationships as edges, not fields?

Influence and access are network properties. Reporting lines, peer trust, and introduction paths form a graph. Fields on Contact can't express many-to-many directed relationships with strength.

NEW RULE — EDGE-RECORD-001: Each Relationship_Edge is a directed link (from_node -> to_node) with type (REPORTS_TO / INFLUENCES / KNOWS / CAN_INTRODUCE) and strength. Nodes may be Contacts or Connectors.

WHY #2 — Why store direction and strength?

A weak acquaintance and a trusted mentor are not interchangeable introduction paths. Direction (who can reach whom) and strength (how warm) determine which path the system should use.

NEW RULE — EDGE-STRENGTH-001: Edges carry direction and strength (warmth/recency/reciprocity). Path-finding to a target prefers high-strength directed paths; weak/cold edges are last resort.

WHY #3 — Why include connectors as nodes in the same graph?

External connectors (Tarabut, KPMG, Backbase MDs) are the warm-intro routes into accounts. Putting them in the same graph lets the system compute the actual shortest warm path from Decimal to any target.

NEW RULE — EDGE-CONNECTOR-001: Connectors are first-class nodes. The system computes warm-introduction paths from Decimal through connectors to target committee members, ranked by total path strength.

WHY #4 — Why track edge decay over time?

Relationships cool. An intro path warm a year ago may be stale. Edges must decay so the system doesn't over-rely on relationships that no longer hold.

NEW RULE — EDGE-DECAY-001: Edge strength decays with inactivity. Stale edges are flagged before being used as introduction paths; using a decayed edge lowers expected path success.

WHY #5 — Deepest reason this entity exists?

It makes relationship capital computable. It exists so the system can answer 'what is our warmest path to the economic buyer?' — turning the Relationship principle into an actual routing algorithm.

NEW RULE — EDGE-PRIMACY-001: Outreach to a high-value target must first check for a warm path via Relationship_Edge. Cold-direct outreach to maximum-tier contacts when a warm path exists is disallowed without justification.

RELATIONSHIP_EDGE — Field Specification

Field	Type	Business Rule	Source
edge_id	UUID	Immutable	EDGE-RECORD-001
from_node / to_node	UUID	Contact or Connector nodes	EDGE-RECORD-001

Field	Type	Business Rule	Source
edge_type	Enum	REPORTS_TO / INFLUENCES / KNOWS / CAN_INTRODUCE	EDGE-RECORD-001
direction / strength	Enum/Float	Routing + warmth	EDGE-STRENGTH-001
last_active_at	Timestamp	Drives decay	EDGE-DECAY-001

ENTITY 14: CONNECTOR

An external person or organization (Tarabut, KPMG alumni, Backbase MD, Geidea) who can provide warm access into target accounts.

WHY #1 — Why are connectors a distinct entity from Contacts?

Connectors play a structural role — they are reusable access assets across many accounts, with their own capital budget and fatigue. A Contact is a target; a Connector is a route. The economics differ.

NEW RULE — CONNECTOR-RECORD-001: Each Connector is stored with reach (which accounts/people they access), relationship strength to Decimal, and a per-connector capital budget. Connectors are reusable access assets.

WHY #2 — Why give each connector a capital budget?

Connector capital is finite (SCARCITY-003). Over-asking burns the relationship. A per-connector quarterly budget with reciprocity tracking prevents fatigue and preserves the asset.

NEW RULE — CONNECTOR-BUDGET-001: Each Connector has a quarterly ask-budget and reciprocity ledger. Exceeding the budget requires giving value back first; fatigue signals pause asks to that connector.

WHY #3 — Why track connector reach explicitly?

The value of a connector is which doors they open. Tarabut reaching SNB/SAB/Alinma simultaneously is worth more than a single-account intro. Reach must be mapped to route requests optimally.

NEW RULE — CONNECTOR-REACH-001: Connector reach maps to accessible accounts and committee members via Relationship_Edge. High-reach connectors are reserved for high-value, multi-account openings.

WHY #4 — Why track reciprocity and value-given?

Connectors are relationships, not vending machines. Sustainable access requires giving before taking. Tracking the give/take balance keeps connector relationships healthy long-term.

NEW RULE — CONNECTOR-RECIPROCITY-001: Each ask debits, each value-delivered credits the reciprocity ledger. Negative reciprocity (taking without giving) blocks further asks until rebalanced.

WHY #5 — Deepest reason Connector exists?

Warm beats cold by an order of magnitude in KSA enterprise selling. The connector is the embodiment of relationship-first GTM — the asset that converts intelligence into access. It exists because trust is transferable through people.

NEW RULE — CONNECTOR-PRIMACY-001: For maximum-tier targets, the system attempts connector-mediated warm entry before any cold approach, subject to connector budget and reciprocity health.

CONNECTOR — Field Specification

Field	Type	Business Rule	Source
connector_id	UUID	Immutable	CONNECTOR-RECORD-001
connector_name / type	String/Enum	PERSON / FIRM / ALUMNI_NETWORK / PARTNER	CONNECTOR-RECORD-001
reach	Object	Accounts/people accessible	CONNECTOR-REACH-001

Field	Type	Business Rule	Source
strength_to_decimal	Float	Relationship warmth	CONNECTOR-RECORD-001
quarterly_ask_budget	Int	Capital cap	CONNECTOR-BUDGET-001
reciprocity_ledger	Object	Give/take balance	CONNECTOR-RECIPROCITY-001

ENTITY 15: MARKET_INTERACTION_HISTORY

The long-memory ledger of every interaction Decimal has had across the market — per person, per account, across time and career moves.

WHY #1 — Why a market-wide history beyond per-account logs?

Reputation is market-wide and person-portable. A person burned at one bank remembers Decimal at the next. Per-account logs can't see cross-account, cross-time reputation.

NEW RULE — MARKETMEM-RECORD-001: Market_Interaction_History aggregates all interactions across accounts and time, keyed to persistent Contact identity, forming Decimal's market reputation memory.

WHY #2 — Why track interactions across career transitions?

MARKETMEM-002 requires knowing if Decimal engaged this PERSON before, anywhere. Career moves are where relationship capital transfers; losing this history loses the relationship.

NEW RULE — MARKETMEM-CAREER-001: When a Contact changes employer, their full interaction history is preserved and surfaced in the new account context. Prior rapport (or damage) carries forward.

WHY #3 — Why compute a reputation blast-radius per contact?

High-influence contacts shape how a whole segment perceives Decimal. A mistake with them propagates. Blast-radius sizing tells the system where maximum care is required.

NEW RULE — MARKETMEM-BLAST-001: Each interaction estimates a reputation blast_radius based on contact influence and network position. High-blast contacts mandate maximum-tier handling and conservative outreach.

WHY #4 — Why track brand-capital depletion signals here?

Repeated negative interactions are an early warning of reputation damage. Aggregating them market-wide surfaces depletion before it becomes a crisis (the Brand Capital Depletion Alert).

NEW RULE — MARKETMEM-DEPLETION-001: Market_Interaction_History monitors depletion signals (rising negative sentiment, ignored outreach, complaints). Two-or-more triggers pause market expansion pending review and repair.

WHY #5 — Deepest reason this entity exists?

It is the system's conscience and long memory — the reason it behaves like a trusted market participant rather than a spam engine. It exists because reputation, once spent, is the hardest capital to rebuild.

NEW RULE — MARKETMEM-PRIMACY-001: Every outreach decision consults Market_Interaction_History for the target person. Prior unresolved damage is a hard gate; the system never re-burns a known-damaged relationship.

MARKET_INTERACTION_HISTORY — Field Specification

Field	Type	Business Rule	Source
history_id	UUID	Immutable ledger entry	MARKETMEM-RECORD-001
contact_id	UUID	Persistent person key	MARKETMEM-CAREER-001
account_context_id	UUID	Account at time of interaction	MARKETMEM-RECORD-001

Field	Type	Business Rule	Source
interaction_summary / sentiment	Object/Enum	What happened + outcome	MARKETMEM-RECORD-001
blast_radius	Object	Reputation propagation estimate	MARKETMEM-BLAST-001
depletion_flags	Array	Brand-capital warning signals	MARKETMEM-DEPLETION-001
occurred_at	Timestamp	Event time	MARKETMEM-RECORD-001

TIER 3 — DECISION ENTITIES

The objects that store HOW the system DECIDES what to do.

ENTITY 16: REASONING_CHAIN

The stored, step-by-step strategic thought process behind any action — the audit trail that makes 'no message without reasoning' enforceable.

WHY #1 — Why store reasoning as an entity, not discard it after generating a message?

Principle 5 mandates a structured reasoning chain before any message. To enforce it, prove it happened, and learn from it, the chain must be persisted, not transient.

NEW RULE — REASON-RECORD-001: Every action-producing reasoning run is stored as a Reasoning_Chain with all mandated steps (signal interpretation, account research, pain inference, persona analysis, strategy decision, generation, QC). Missing steps invalidate the output.

WHY #2 — Why store each step separately rather than a final rationale?

Failure could occur at any step (wrong pain inference, wrong persona). Step-level storage lets the system locate exactly where reasoning went wrong and improve that step.

NEW RULE — REASON-STEPS-001: Each reasoning step stores its input, output, and confidence. A weak/low-confidence step flags the whole chain for review; partial chains are rejected (no shortcuts).

WHY #3 — Why link the chain to the signals and intelligence it used?

Reasoning built on stale or unsourced data is unsafe. Linking inputs lets the system invalidate conclusions when underlying data changes or expires.

NEW RULE — REASON-INPUTS-001: Each chain links the specific Signals, Enrichment, and entity states it consumed. If a linked input later expires or is corrected, dependent chains are flagged for re-reasoning.

WHY #4 — Why record confidence and hallucination checks in the chain?

Hallucination is a named failure mode. The chain must carry QC results so unsafe outputs are caught before send and so reviewer trust is calibrated.

NEW RULE — REASON-QC-001: Each chain records QC results: hallucination_score, factual-grounding check, tone/enterprise-suitability. Chains failing QC thresholds cannot produce a sendable Message.

WHY #5 — Deepest reason this entity exists?

It is the difference between an intelligent system and a template engine. It exists to make every action explainable, defensible, and improvable — the physical form of 'reasoning before messaging.'

NEW RULE — REASON-PRIMACY-001: No Message, Outreach_Sequence step, or strategic action may exist without a linked, complete, QC-passing Reasoning_Chain. This is enforced at the data layer.

REASONING_CHAIN — Field Specification

Field	Type	Business Rule	Source
reasoning_chain_id	UUID	Immutable	REASON-RECORD-001
account_id / contact_id	UUID	Subject of reasoning	—
steps	Array[Object]	Each step: input, output, confidence	REASON-STEP-S-001
input_links	Object	Signals/Enrichment/state consumed	REASON-INPUT-S-001
qc_results	Object	Hallucination, grounding, tone	REASON-QC-001
status	Enum	COMPLETE / PARTIAL_REJECTED / QC_FAILED	REASON-PRIMACY-001
created_at	Timestamp	Run time	—

ENTITY 17: OUTREACH_SEQUENCE

A planned multi-touch engagement plan for a contact/account: ordered steps, channels, timing gaps, and stop conditions.

WHY #1 — Why model a sequence as an entity, not just send messages?

Enterprise engagement is multi-touch over time with hard pacing constraints. The sequence is the plan that enforces cadence, channel mix, and limits — properties no single message holds.

NEW RULE — SEQ-RECORD-001: Each Outreach_Sequence stores ordered steps, channel per step, timing gaps, and stop conditions. It enforces hard limits: max touches per channel, minimum gaps, and the 14/10-day spreads.

WHY #2 — Why encode stop/exit conditions in the sequence?

A sequence that ignores responses is spam. It must halt on reply, on negative sentiment, or on window closure — exit logic is core, not optional.

NEW RULE — SEQ-EXIT-001: Each sequence defines exit conditions (reply received, negative sentiment, window closed, do-not-contact). Triggering any exit immediately pauses/terminates remaining steps.

WHY #3 — Why bind sequence pacing to system-wide rate limits?

Per-contact limits aren't enough; the system has global throughput caps (max new accounts/day in MVP). The sequence must respect both contact-level and system-level pacing.

NEW RULE — SEQ-RATELIMIT-001: Sequence scheduling respects contact-level limits AND global throughput caps. When global capacity is exhausted, sequence starts queue by priority (Decision Score), not first-come.

WHY #4 — Why make each step re-evaluate before firing?

State changes between planning and execution (window closes, contact leaves, sentiment drops). A step must re-check conditions at fire-time, not blindly execute the plan.

NEW RULE — SEQ-REEVAL-001: Before each step fires, the system re-evaluates current account/contact/window state and the governing Reasoning_Chain freshness. Stale or invalidated steps are regenerated or cancelled.

WHY #5 — Deepest reason this entity exists?

It operationalizes timing and restraint — turning strategy into paced, condition-aware action instead of a burst of messages. It exists because in enterprise, cadence and patience are as important as content.

NEW RULE — SEQ-PRIMACY-001: All outbound activity flows through an Outreach_Sequence governed by these constraints. Ad-hoc out-of-sequence sends are prohibited except explicit human-initiated one-offs, which are still logged.

OUTREACH_SEQUENCE — Field Specification

Field	Type	Business Rule	Source
sequence_id	UUID	Immutable	SEQ-RECORD-001
account_id / contact_id	UUID	Target	—
steps	Array[Object]	Order, channel, gap, planned content ref	SEQ-RECORD-001
exit_conditions	Array	Halt triggers	SEQ-EXIT-001
status	Enum	ACTIVE / PAUSED / COMPLETED / EXITED	SEQ-EXIT-001
priority_score	Float	Decision Score for queue ordering	SEQ-RATELIMIT-001
created_at / updated_at	Timestamp	System-managed	—

ENTITY 18: ACCOUNT_STRATEGY

The system's standing plan for an account: chosen angle, competitive posture, target committee members, connector path, and readiness gates.

WHY #1 — Why have a standing strategy entity above individual sequences?

Sequences are tactics; the account needs a coherent strategy they all serve. Without it, touches contradict each other and the account experiences an incoherent Decimal.

NEW RULE — STRAT-RECORD-001: Each engaged account has one active Account_Strategy stating the value angle, competitive posture, target committee members, and connector path. Sequences must align to it.

WHY #2 — Why tie strategy to a specific initiative and window?

Strategy without a concrete initiative and timing is abstract. Anchoring it makes the plan actionable and time-aware.

NEW RULE — STRAT-ANCHOR-001: Account_Strategy anchors to a target Initiative and any Buying_Window. Absent an initiative, strategy defaults to discovery/relationship-building, not solution-selling.

WHY #3 — Why record the readiness gates in the strategy?

Strategic Patience requires not engaging before Decimal is ready (references, collateral, compliance). The strategy must hold the gates that must clear before active selling.

NEW RULE — STRAT-READINESS-001: Account_Strategy records readiness gates (e.g. SAMA CSF brief, data residency, Islamic-finance support) that must clear before solution-selling. Unmet hard gates hold the strategy in preparation mode.

WHY #4 — Why version strategy and require review on major state change?

Accounts change (sponsor leaves, incumbent renews). A strategy frozen against old reality fails. Versioning and triggered review keep strategy live.

NEW RULE — STRAT-VERSION-001: Strategy is versioned; major events (sponsor departure, window close, competitive renewal) trigger mandatory strategy review. Superseded versions are retained for learning.

WHY #5 — Deepest reason this entity exists?

It is the account-level intelligence that makes Decimal feel like one coherent, strategic partner rather than a stream of disconnected messages. It exists because account-centric selling needs an account-centric plan.

NEW RULE — STRAT-PRIMACY-001: Every Outreach_Sequence and Message must reference and conform to the active Account_Strategy. Conflicts between a tactic and the strategy are resolved in favor of the strategy or escalate to review.

ACCOUNT_STRATEGY — Field Specification

Field	Type	Business Rule	Source
strategy_id	UUID	Immutable version node	STRAT-RECORD-001
account_id	UUID	One active per account	STRAT-RECORD-001
value_angle / competitive_posture	Object/Enum	Core plan + incumbency stance	STRAT-RECORD-001
target_bcm_ids / connector_path	Array[UUID]	Who + how to reach	STRAT-RECORD-001
anchor_initiative_id / window_id	UUID	Initiative + timing	STRAT-ANCHOR-001
readiness_gates	Array[Object]	Gates to clear before selling	STRAT-READINESS-001
version / status	Int/Enum	PREPARATION / ACTIVE / SUPERSEDED	STRAT-VERSION-001

ENTITY 19: PORTFOLIO_ALLOCATION

The system-wide decision of how to spend finite capital (attention, connector, brand) across all accounts in a cycle.

WHY #1 — Why a portfolio entity above per-account decisions?

Capital is finite and shared. Each account optimizing alone overspends the whole. A portfolio view allocates scarce capacity across accounts by expected return.

NEW RULE — PORT-RECORD-001: Each allocation cycle produces a Portfolio_Allocation distributing finite capital (attention slots, connector asks, daily account starts) across accounts ranked by Decision Score and EROA.

WHY #2 — Why rank by Expected Return on Attention, not raw score?

A high score that costs enormous capital may be worse than a moderate score that's cheap. EROA normalizes opportunity by cost, maximizing total return under constraint.

NEW RULE — PORT-EROA-001: Allocation ranks accounts by EROA = Effective Opportunity / Total Capital Cost. Negative-EROA accounts are excluded this cycle regardless of raw attractiveness.

WHY #3 — Why enforce diversification / concentration limits?

All capital on one whale risks total loss if it stalls. Some spread protects the portfolio and the brand. Concentration caps manage that risk.

NEW RULE — PORT-DIVERSIFY-001: Allocation enforces concentration limits so no single account or connector consumes beyond a cap of cycle capital. Over-concentration requires explicit human approval.

WHY #4 — Why reconcile allocation against actual spend each cycle?

Plans drift from reality. Comparing planned vs actual capital spend catches overspend and feeds better future allocation.

NEW RULE — PORT-RECONCILE-001: Each cycle reconciles planned vs actual capital spend per account/connector. Systematic overspend triggers recalibration of cost models.

WHY #5 — Deepest reason this entity exists?

It embodies scarcity economics — the recognition that attention is the binding constraint and must be invested, not sprayed. It exists so the system behaves like a disciplined investor of finite capital.

NEW RULE — PORT-PRIMACY-001: No account may consume system capital outside the active Portfolio_Allocation. Sequence starts and connector asks draw down allocated budgets; exhausted budgets queue work to the next cycle.

PORTFOLIO_ALLOCATION — Field Specification

Field	Type	Business Rule	Source
allocation_id	UUID	Immutable per cycle	PORT-RECORD-001
cycle_period	Date range	Allocation window	PORT-RECORD-001
account_allocations	Array[Object]	Per-account capital + EROA	PORT-EROA-001
concentration_limits	Object	Caps per account/connector	PORT-DIVERSIFY-001
reconciliation	Object	Planned vs actual spend	PORT-RECONCILE-001
created_at	Timestamp	Cycle start	—

ENTITY 20: CONFLICT_RESOLUTION

A recorded instance of two rules conflicting and how it was resolved, building a precedent library over time.

WHY #1 — Why record rule conflicts as entities?

Rules will conflict (timing says act now, readiness says wait). The system needs a deterministic resolution AND a memory of how it resolved, so similar conflicts resolve consistently.

NEW RULE — CONFLICT-RECORD-001: Each rule conflict is recorded with the conflicting rules, context, the tier-hierarchy resolution applied, and the outcome. Silent resolution is prohibited.

WHY #2 — Why resolve via the 6-tier hierarchy rather than ad hoc?

Ad hoc resolution is unpredictable and unauditable. The Constraint Hierarchy (Legal > Safety > Relationship > Timing > Revenue > Optimization) gives a stable, explainable ordering.

NEW RULE — CONFLICT-HIERARCHY-001: Conflicts resolve by the 6-tier Constraint Hierarchy. The winning tier and rule are recorded. Conflicts unresolvable within time limits (4h/24h) escalate to human review.

WHY #3 — Why build a precedent library from resolutions?

Repeated identical conflicts shouldn't be re-litigated. After enough consistent resolutions, the pattern becomes a standing rule, speeding future decisions.

NEW RULE — CONFLICT-PRECEDENT-001: Three-or-more consistent resolutions of the same conflict type promote to a standing rule in Rule_Registry. The precedent library is queried before escalating new conflicts.

WHY #4 — Why time-box conflict resolution?

An unresolved conflict stalls action and can miss windows. Hard limits force either automated resolution or human escalation, preventing decision paralysis.

NEW RULE — CONFLICT-TIMEBOX-001: Conflicts must resolve within tiered limits (4h automated / 24h with human). Breaching the limit auto-escalates; timing-critical conflicts get the shortest limit.

WHY #5 — Deepest reason this entity exists?

It makes the system's value hierarchy explicit and consistent — ensuring that when principles collide, safety and relationships reliably outrank short-term revenue. It exists so the system's character is stable under pressure.

NEW RULE — CONFLICT-PRIMACY-001: Any decision arising from a resolved conflict carries a link to its Conflict_Resolution record, making the trade-off auditable. Revenue may never silently override Legal, Safety, or Relationship tiers.

CONFLICT_RESOLUTION — Field Specification

Field	Type	Business Rule	Source
conflict_id	UUID	Immutable	CONFLICT-RECORD-001

Field	Type	Business Rule	Source
conflicting_rule_ids	Array	Rules in tension	CONFLICT-RECORD-001
context	Object	Account/decision context	CONFLICT-RECORD-001
resolution_tier / winning_rule	Enum/ID	Hierarchy outcome	CONFLICT-HIERARCHY-001
escalated	Bool	Human review required	CONFLICT-TIMEBOX-001
promoted_to_standing_rule	Bool	Precedent formed	CONFLICT-PRECEDENT-001
resolved_at	Timestamp	Resolution time	—

ENTITY 21: DECISION_PATTERN

A learned, reusable pattern: 'in situations like X, decision Y tends to produce outcome Z' — the system's accumulating playbook.

WHY #1 — Why store decision patterns as entities?

Repeated decisions shouldn't be reasoned from scratch each time. Capturing successful (and failed) decision patterns lets the system reuse what works and avoid what doesn't.

NEW RULE — PATTERN-RECORD-001: Recurring decision contexts and their outcomes are abstracted into Decision_Pattern entities (situation signature, decision taken, outcome distribution, confidence).

WHY #2 — Why require an outcome distribution, not a single result?

One success can be luck. A pattern is trustworthy only across many instances. Storing the distribution captures reliability, not anecdote.

NEW RULE — PATTERN-EVIDENCE-001: Each pattern carries the count and outcome distribution of instances. Patterns below a minimum instance threshold are advisory only and cannot auto-drive decisions.

WHY #3 — Why link patterns to the rules they support or challenge?

Patterns are how rules are validated empirically. A pattern that contradicts a rule is a signal the rule needs revisiting; one that confirms it strengthens confidence.

NEW RULE — PATTERN-RULE-LINK-001: Patterns link to related Rule_Registry entries. A pattern persistently contradicting a rule flags that rule for review; confirming patterns raise rule confidence.

WHY #4 — Why decay or retire stale patterns?

Markets change; yesterday's winning pattern can become today's mistake. Patterns must age out or be re-validated to avoid the system fighting the last war.

NEW RULE — PATTERN-DECAY-001: Pattern confidence decays without fresh confirming instances. Stale patterns are demoted to advisory and re-validated before reuse in automated decisions.

WHY #5 — Deepest reason this entity exists?

It is how the system gets wiser over time rather than merely busier — the accumulation of strategic experience. It exists to turn outcomes into reusable intelligence, the essence of a learning system.

NEW RULE — PATTERN-PRIMACY-001: Decision patterns inform but never silently override explicit rules or human judgment. Auto-application requires sufficient evidence and current confidence; otherwise patterns are presented as recommendations.

DECISION_PATTERN — Field Specification

Field	Type	Business Rule	Source
pattern_id	UUID	Immutable	PATTERN-REC ORD-001
situation_signature	Object	Context fingerprint	PATTERN-REC ORD-001
decision_taken	Object	The recurring decision	PATTERN-REC ORD-001
outcome_distribution	Object	Instance count + results	PATTERN-EVID ENCE-001
related_rule_ids	Array	Rules supported/challenged	PATTERN-RULE -LINK-001
confidence / last_confirmed_at	Float/Timestamp	Decay tracking	PATTERN-DEC AY-001

ENTITY 22: RISK_ASSESSMENT

A pre-action evaluation of what could go wrong with a proposed decision, feeding the Risk Modifier in the opportunity equation.

WHY #1 — Why assess risk as a distinct entity before acting?

Opportunity attractiveness is not enough; a high-reward move with high reputational risk may be wrong. Explicit risk assessment lets the system weigh downside, not just upside.

NEW RULE — RISK-RECORD-001: Before significant actions, a Risk_Assessment evaluates reputational, relationship, compliance, and timing risks, producing a Risk Modifier consumed by the Effective Opportunity equation.

WHY #2 — Why separate risk types rather than one score?

Compliance risk and tone risk demand different mitigations. A blended score hides which risk dominates and how to address it.

NEW RULE — RISK-TYPED-001: Risk is decomposed by type (reputational, relationship, compliance/legal, timing, execution). Any single type breaching a hard threshold blocks the action regardless of blended score.

WHY #3 — Why tie compliance risk to a hard gate?

In KSA banking, compliance (SAMA CSF, data residency, Shariah) is a binary gate — failure ends the conversation. Compliance risk cannot be traded off against opportunity.

NEW RULE — RISK-COMPLIANCE-001: Compliance/legal risk is a hard gate (Tier 1 of the Constraint Hierarchy). Any unresolved compliance risk blocks action; it can never be outweighed by opportunity size.

WHY #4 — Why record mitigations and residual risk?

Risks can often be mitigated (warm intro reduces cold-outreach risk). Recording mitigation and residual risk shows whether the action became acceptable and why.

NEW RULE — RISK-MITIGATION-001: Each assessment records available mitigations and resulting residual risk. Actions proceed only if residual risk is within tier limits; otherwise they hold or escalate.

WHY #5 — Deepest reason this entity exists?

It encodes prudence — the discipline of asking 'what's the worst case?' before spending irreplaceable brand and relationship capital. It exists so the system protects the downside, not just chases the upside.

NEW RULE — RISK-PRIMACY-001: High-blast-radius or maximum-tier-contact actions require a passing Risk_Assessment. Bypassing risk assessment for such actions is prohibited.

RISK_ASSESSMENT — Field Specification

Field	Type	Business Rule	Source
risk_assessment_id	UUID	Immutable	RISK-RECORD-001
subject_action_ref	UUID	Decision/message under review	RISK-RECORD-001
risk_by_type	Object	Reputational/relationship/compliance/timing/execution	RISK-TYPED-001
compliance_gate_status	Enum	PASS / FAIL / UNRESOLVED	RISK-COMPLIANCE-001
mitigations / residual_risk	Object	Applied + remaining	RISK-MITIGATION-001
risk_modifier	Float	Feeds opportunity equation	RISK-RECORD-001

ENTITY 23: HUMAN_REVIEW_RECORD

A record of a human reviewing, approving, editing, or rejecting an AI decision — the audit trail of AI-human collaboration.

WHY #1 — Why record human reviews as entities?

AI-human collaboration (Principle 3) requires that human judgment be captured, not lost. The review record proves oversight happened and what the human decided.

NEW RULE — REVIEW-RECORD-001: Each human review is stored with the artifact reviewed, the decision (approve/edit/reject), edits made, rationale, and reviewer identity. Reviews are immutable once submitted.

WHY #2 — Why capture the rationale, not just the verdict?

A bare reject teaches the system nothing. The rationale is the training signal that improves future AI proposals and detects rubber-stamping.

NEW RULE — REVIEW-RATIONALE-001: Approvals and rejections require a rationale. Rationale-free verdicts are flagged; high rates of rationale-free approvals signal rubber-stamping and trigger oversight review.

WHY #3 — Why monitor for rubber-stamping explicitly?

Rubber-stamping is a named failure mode that hollows out oversight. The system must detect reviewers who approve everything instantly without engagement.

NEW RULE — REVIEW-RUBBERSTAMP-001: Review patterns (approval rate, review latency, edit frequency) are monitored. Anomalous rubber-stamping patterns reduce the effective trust granted to that reviewer's approvals and alert governance.

WHY #4 — Why feed reviews back into AI confidence and autonomy?

Trust must be earned and calibrated. Consistent human agreement should expand AI autonomy on that decision type; disagreement should contract it.

NEW RULE — REVIEW-TRUST-001: Review outcomes update the AI's earned-autonomy level per decision type via the Trust_Capital_Ledger. Sustained agreement raises autonomy; disagreement lowers it and may reinstate mandatory review.

WHY #5 — Deepest reason this entity exists?

It is the hinge of human-AI collaboration — the mechanism by which authority is shared, trust is calibrated, and accountability is preserved. It exists so the human remains meaningfully in command.

NEW RULE — REVIEW-PRIMACY-001: Decisions above defined risk/autonomy thresholds require a Human_Review_Record before execution. The system never self-grants authority beyond its earned, reviewed autonomy level.

HUMAN_REVIEW_RECORD — Field Specification

Field	Type	Business Rule	Source
review_id	UUID	Immutable	REVIEW-RECORD-001
artifact_ref	UUID	Reasoning_Chain/Message/Strategy reviewed	REVIEW-RECORD-001
reviewer_id	UUID	Human identity	REVIEW-RECORD-001
decision	Enum	APPROVE / EDIT / REJECT	REVIEW-RECORD-001
edits / rationale	Object/Text	Changes + reasoning	REVIEW-RATIONALE-001
review_latency	Duration	Rubber-stamp monitoring	REVIEW-RUBBERSTAMP-001
trust_impact	Object	Autonomy adjustment	REVIEW-TRUST-001

TIER 4 — GOVERNANCE ENTITIES

The objects that store HOW THE SYSTEM GOVERNS ITSELF.

ENTITY 24: TRUST_CAPITAL_LEDGER

The running account of trust the AI has earned per decision type, governing how much autonomy it is granted.

WHY #1 — Why track AI trust as a ledger?

Autonomy should be earned and revocable, not fixed. A ledger of trust per decision type lets autonomy expand with proven reliability and contract after failures.

NEW RULE — TRUST-RECORD-001: The Trust_Capital_Ledger tracks earned-autonomy balance per decision type, credited by successful reviewed decisions and debited by failures/overrides. Autonomy levels derive from this balance.

WHY #2 — Why per decision type rather than one global trust level?

The AI may be reliable at signal interpretation but weak at competitive strategy. One global number over- or under-grants autonomy. Per-type trust calibrates correctly.

NEW RULE — TRUST-TYPED-001: Trust is tracked per decision type. High trust in one type does not transfer to another. Each type has its own threshold for moving from mandatory-review to autonomous action.

WHY #3 — Why debit trust on failures and overrides?

Trust must be responsive to evidence. A failure or a human override is evidence the AI was wrong; trust (and autonomy) should drop accordingly.

NEW RULE — TRUST-DEBIT-001: Failures, human rejections, and bad outcomes debit trust for that decision type. A sharp debit can instantly reinstate mandatory human review (autonomy circuit-breaker).

WHY #4 — Why guard against trust gaming / metric manipulation?

If the AI could inflate its own trust (e.g. by routing only easy decisions for review), oversight collapses. The ledger must resist manipulation (a named failure mode).

NEW RULE — TRUST-INTEGRITY-001: Trust crediting uses outcome-weighted, sampled, and difficulty-adjusted decisions; the AI cannot self-select which decisions count toward trust. Manipulation patterns freeze autonomy and alert governance.

WHY #5 — Deepest reason this entity exists?

It is the economic engine of human-AI collaboration — making autonomy a function of demonstrated reliability rather than assumption. It exists so the system earns its independence honestly.

NEW RULE — TRUST-PRIMACY-001: No autonomous action may exceed the earned-autonomy level recorded in the Trust_Capital_Ledger for that decision type. The ledger is the single source of truth for AI authority.

TRUST_CAPITAL_LEDGER — Field Specification

Field	Type	Business Rule	Source
ledger_id	UUID	Immutable entries	TRUST-RECORD-001
decision_type	Enum	Per-type trust bucket	TRUST-TYPE-001
balance / autonomy_level	Float/Enum	Earned trust + granted autonomy	TRUST-RECORD-001
credit_debit_events	Array[Object]	Outcome-weighted changes	TRUST-DEBIT-001
integrity_flags	Array	Manipulation detection	TRUST-INTEGRITY-001
updated_at	Timestamp	System-managed	—

ENTITY 25: SYSTEM_EVENT

The immutable, append-only log of everything the system does — the spine of auditability and event-driven coordination.

WHY #1 — Why a dedicated system-event log entity?

Event-driven architecture (Principle 1) and enterprise auditability both require a complete, immutable record of every state change and action. Scattered logs can't provide a single coherent timeline.

NEW RULE — SYSEVENT-RECORD-001: Every significant state change, decision, and action emits an immutable, append-only System_Event with actor, type, payload, and timestamp. The log is never edited or deleted.

WHY #2 — Why make it append-only and immutable?

Audit integrity demands tamper-evidence. If events can be altered, the audit trail is worthless for compliance and debugging.

NEW RULE — SYSEVENT-IMMUTABLE-001: System_Events are write-once. Corrections are new compensating events referencing the original, never edits. This preserves a tamper-evident history for enterprise/compliance audit.

WHY #3 — Why use events to coordinate, not just to log?

In an event-driven system, events are the coordination mechanism — scoring retriggers, window openings, and sequence advances are all event-driven. The log doubles as the nervous system.

NEW RULE — SYSEVENT-COORD-001: Designated System_Events trigger downstream workflows (scoring recalculation, window evaluation, sequence re-eval). Coordination flows through events, ensuring reactions are traceable to causes.

WHY #4 — Why include causal linkage between events?

To debug and explain, you must follow cause to effect. Linking each event to its triggering event reconstructs full decision chains across the system.

NEW RULE — SYSEVENT-CAUSAL-001: Each event records its triggering_event_id where applicable, forming causal chains. Any action can be traced backward to the originating signal and forward to its consequences.

WHY #5 — Deepest reason this entity exists?

It is the system's memory and conscience combined — the ground truth that everything else reconciles against. It exists so the system is fully explainable, recoverable, and trustworthy under audit.

NEW RULE — SYSEVENT-PRIMACY-001: Any denormalized cache (e.g. Account.current_state) must be reconstructable from the System_Event log. In any discrepancy, the event log is authoritative.

SYSTEM_EVENT — Field Specification

Field	Type	Business Rule	Source
event_id	UUID	Immutable, append-only	SYSEVENT-RECORD-001
actor	Enum	SYSTEM / AI / HUMAN / EXTERNAL	SYSEVENT-RECORD-001
event_type	String	Categorized action/state change	SYSEVENT-RECORD-001
payload	Object	State delta / details	SYSEVENT-RECORD-001
triggering_event_id	UUID	Causal linkage	SYSEVENT-CASUAL-001
occurred_at	Timestamp	Event time	SYSEVENT-RECORD-001

ENTITY 26: RULE_REGISTRY

The versioned catalog of every business rule, meta-principle, weight-set, and threshold the system enforces.

WHY #1 — Why store the rules themselves as data?

The system enforces ~230 operational rules plus 25 meta-principles. Hard-coding them makes change impossible to audit or roll back. As versioned data, rules become governable.

NEW RULE — RULEREG-RECORD-001: Every rule, meta-principle, threshold, and weight-set is stored in the Rule_Registry with id, text, tier, version, and active status. The engine reads rules from the registry, not from code constants.

WHY #2 — Why version rules and retain history?

Rules evolve (monthly recalibration). To compare past decisions fairly and to roll back bad changes, every rule version must be retained.

NEW RULE — RULEREG-VERSION-001: Rule changes create new versions; prior versions are retained. Decisions reference the rule version in force at decision time, enabling valid historical audit and rollback.

WHY #3 — Why classify each rule by constraint tier?

The 6-tier Constraint Hierarchy resolves conflicts. Each rule must carry its tier so conflict resolution is deterministic.

NEW RULE — RULEREG-TIER-001: Each rule is tagged with its Constraint Hierarchy tier (Legal/Safety/Relationship/Timing/Revenue/Optimization). Tier drives conflict resolution and which rules are immutable meta-principles vs mutable operational rules.

WHY #4 — Why distinguish immutable meta-principles from mutable rules?

Some principles (legal, safety, core values) must never be auto-changed; others (weights, thresholds) should adapt. The registry must enforce that distinction.

NEW RULE — RULEREG-IMMUTABLE-001: The 25 meta-principles are flagged immutable and cannot be altered by automated recalibration or pattern promotion — only by explicit governed human change. Operational rules may adapt within bounds.

WHY #5 — Deepest reason this entity exists?

It is the codified character of the system — its values and judgment made explicit, versioned, and governable. It exists so the system's behavior can evolve without ever silently drifting from its principles.

NEW RULE — RULEREG-PRIMACY-001: No rule may govern a decision unless it exists, is active, and is tier-classified in the Rule_Registry. Conflict_Resolution and Decision_Pattern promotions write here under governance control.

RULE_REGISTRY — Field Specification

Field	Type	Business Rule	Source
rule_id	String	Stable identifier (e.g. SIGNAL-DECAY-001)	RULEREG-RECORD-001
text / parameters	Text/Object	Rule content + thresholds/weights	RULEREG-RECORD-001
tier	Enum	Constraint Hierarchy tier	RULEREG-TIER-001
version / active	Int/Bool	Version history + status	RULEREG-VERSION-001
immutable	Bool	Meta-principle protection	RULEREG-IMMUTABLE-001
effective_from / retired_at	Timestamp	Temporal validity	RULEREG-VERSION-001

ENTITY 27: AI_MODEL_REGISTRY

The catalog of AI models in use, their versions, capabilities, and performance — enabling rollback and accountability.

WHY #1 — Why register AI models as entities?

Outputs depend on which model produced them. To attribute outcomes, compare models, and roll back a bad upgrade, each model and version must be a tracked asset.

NEW RULE — MODELREG-RECORD-001: Each AI model and version is registered with capabilities, intended decision types, and activation status. Generated artifacts record the model_id that produced them.

WHY #2 — Why track per-model performance?

A model upgrade can silently degrade quality. Per-model performance metrics catch regressions and inform which model handles which task.

NEW RULE — MODELREG-PERF-001: Performance metrics (quality, hallucination rate, outcome correlation) are tracked per model/version. Regression beyond thresholds blocks promotion and can trigger rollback.

WHY #3 — Why route decision types to specific models?

Different tasks suit different models (cheap/fast for parsing, strong for strategy). Explicit routing optimizes cost and quality.

NEW RULE — MODELREG-ROUTING-001: Decision types route to designated models per the registry. Routing changes are versioned and auditable; a model may only handle decision types it is approved for.

WHY #4 — Why require rollback capability?

If a new model misbehaves in production, the system must revert instantly. Rollback is a safety requirement, not a nicety.

NEW RULE — MODELREG-ROLLBACK-001: Every model activation is reversible. A one-action rollback restores the prior approved model for affected decision types; rollback events are logged as System_Events.

WHY #5 — Deepest reason this entity exists?

It keeps the intelligence behind the system accountable and controllable rather than an opaque, unversioned black box. It exists so model change is governed like any other production change.

NEW RULE — MODELREG-PRIMACY-001: No AI generation may use an unregistered or unapproved model for a given decision type. Reasoning_Chains and Messages must reference a valid AI_Model_Registry entry.

AI_MODEL_REGISTRY — Field Specification

Field	Type	Business Rule	Source
model_id	String	Model + version identifier	MODELREG-RECORD-001
capabilities / approved_decision_types	Object/Array	What it may do	MODELREG-ROUTING-001
performance_metrics	Object	Quality/hallucination/outcome	MODELREG-PERF-001
active / rollback_target	Bool/ID	Status + revert pointer	MODELREG-ROLLBACK-001
activated_at	Timestamp	Activation time	—

ENTITY 28: PROMPT_VERSION

A versioned prompt template used to generate AI outputs, enabling optimization, A/B testing, and rollback.

WHY #1 — Why version prompts as entities?

Prompts shape outputs as much as models do. To optimize messaging and trace why outputs changed, each prompt version must be a tracked artifact.

NEW RULE — PROMPT-RECORD-001: Each prompt template version is stored with its text, target decision type, and model compatibility. Generated artifacts record prompt_version for traceability.

WHY #2 — Why tie outcomes back to prompt versions?

To know which phrasing converts, outcomes must attribute to the prompt that produced them. Without this link, prompt optimization is guesswork.

NEW RULE — PROMPT-OUTCOME-001: Outcomes (response rate, sentiment, conversion) attribute to the prompt_version via Message and Outcome_Attribution, enabling evidence-based prompt optimization.

WHY #3 — Why support A/B testing of prompts?

Improvement requires controlled comparison. The entity must allow concurrent versions with traffic splits and significance tracking.

NEW RULE — PROMPT-AB-001: Prompt versions support controlled A/B allocation with sample-size and significance tracking. Winning variants promote only on statistically meaningful improvement, not noise.

WHY #4 — Why require rollback and approval for prompt changes?

A bad prompt can damage brand at scale instantly. Changes need approval and instant revert, like any production change touching customers.

NEW RULE — PROMPT-GOVERNANCE-001: Prompt promotion to production requires approval and is reversible. Prompts affecting maximum-tier contacts undergo human review before activation.

WHY #5 — Deepest reason this entity exists?

It treats the system's words — its actual voice to the market — as a governed, improvable, accountable asset. It exists because in this system, language is the product surface, and it must be controlled.

NEW RULE — PROMPT-PRIMACY-001: No Message may be generated by an unregistered prompt. Every AI-generated artifact links to a valid Prompt_Version; untracked prompt use is prohibited.

PROMPT_VERSION — Field Specification

Field	Type	Business Rule	Source
prompt_version_id	String	Version identifier	PROMPT-RECORD-001
template_text / decision_type	Text/Enum	Prompt + purpose	PROMPT-RECORD-001
model_compatibility	Array	Models it works with	PROMPT-RECORD-001
outcome_metrics	Object	Attributed performance	PROMPT-OUTCOME-001
ab_allocation	Object	Traffic split + significance	PROMPT-AB-001
active / approved_by	Bool/UUID	Governance status	PROMPT-GOVERNANCE-001

ENTITY 29: CAPABILITY_GAP

A recorded instance where the system recognized it lacked the data, skill, or authority to act well — institutionalized humility.

WHY #1 — Why record what the system CANNOT do as an entity?

A system that hides its blind spots is dangerous. Capturing gaps where it lacked data, capability, or confidence turns unknowns into a managed backlog rather than silent failures.

NEW RULE — GAP-RECORD-001: When the system encounters a decision it cannot make confidently (missing data, unsupported case, low model competence), it records a Capability_Gap instead of guessing.

WHY #2 — Why prefer recording a gap over a low-confidence guess?

A confident-sounding wrong answer damages trust and brand. Declaring a gap and escalating is safer than fabricating (anti-hallucination).

NEW RULE — GAP-ESCALATE-001: Recognized gaps route the decision to human review or hold, never to a fabricated answer. The system explicitly surfaces 'I don't know / can't yet' rather than hallucinating.

WHY #3 — Why aggregate gaps into a roadmap?

Recurring gaps reveal where to invest (new data source, new capability, readiness gap to close). Aggregation turns individual misses into prioritized improvement.

NEW RULE — GAP-ROADMAP-001: Recurring Capability_Gaps aggregate into an improvement/readiness roadmap with frequency and business-impact ranking, feeding investment and Strategic Readiness planning.

WHY #4 — Why link gaps to readiness and patience?

Many gaps are readiness gaps (no reference, no compliance brief). Linking them connects 'we can't act well' to 'we're not ready to engage,' enforcing strategic patience.

NEW RULE — GAP-READINESS-001: Capability_Gaps tied to engagement readiness feed the Strategic Readiness Score. Unclosed critical gaps hold engagement until addressed (the Patience Test).

WHY #5 — Deepest reason this entity exists?

It institutionalizes humility — the system's willingness to admit limits, which is the foundation of trust and safety. It exists so the system improves deliberately instead of failing silently.

NEW RULE — GAP-PRIMACY-001: Declaring a Capability_Gap is always permitted and never penalized. Acting confidently outside demonstrated capability is the failure mode the system is built to prevent.

CAPABILITY_GAP — Field Specification

Field	Type	Business Rule	Source
gap_id	UUID	Immutable	GAP-RECORD-001
context	Object	Decision the system couldn't make	GAP-RECORD-001
gap_type	Enum	DATA / CAPABILITY / AUTHORITY / READINESS	GAP-RECORD-001
resolution_path	Enum	HUMAN_REVIEW / HOLD / ROADMAP	GAP-ESCALATE-001
frequency / business_impact	Int/Object	Roadmap ranking	GAP-ROADMAP-001
readiness_link	UUID	Tie to readiness gate	GAP-READINESS-001

TIER 5 — ANALYTICS ENTITIES

The objects that store WHAT THE SYSTEM LEARNS.

ENTITY 30: OUTCOME_ATTRIBUTION

The record linking outcomes (replies, meetings, pilots, deals) back to the specific decisions, reasoning, prompts, and timing that caused them.

WHY #1 — Why attribute outcomes as a dedicated entity?

Learning requires knowing what worked and why. Attribution ties results back to the decisions that produced them, turning outcomes into improvement signal.

NEW RULE — ATTR-RECORD-001: Each meaningful outcome (reply, meeting, pilot, win/loss) is linked to the contributing Reasoning_Chain, Prompt_Version, timing/window, and connector path that produced it.

WHY #2 — Why model multi-touch attribution rather than last-touch?

Enterprise wins come from many touches over time. Last-touch attribution misleads; a fair model credits the sequence that built the outcome.

NEW RULE — ATTR-MULTITOUCH-001: Attribution uses a multi-touch model crediting the contributing touches/decisions across the journey, not only the final action. Credit weights are explicit and auditable.

WHY #3 — Why attribute failures as well as successes?

Avoiding mistakes is as valuable as repeating wins. Loss attribution reveals which decisions, timings, or angles destroy opportunities.

NEW RULE — ATTR-FAILURE-001: Negative outcomes (opt-outs, negative sentiment, losses) are attributed with equal rigor. Failure attribution feeds Decision_Pattern decay and Rule_Registry review.

WHY #4 — Why feed attribution into scoring weights and patterns?

Attribution is only useful if it changes behavior. Its output must recalibrate scoring weights, timing models, and decision patterns.

NEW RULE — ATTR-FEEDBACK-001: Attribution results feed Feedback_Loop_Entries that recalibrate scoring weights, timing models, persona inferences, and pattern confidence. Attribution without feedback is prohibited as wasted learning.

WHY #5 — Deepest reason this entity exists?

It closes the loop from action to result to improvement — the mechanism that makes the system measurably better over time rather than merely active. It exists because unattributed activity teaches nothing.

NEW RULE — ATTR-PRIMACY-001: Every significant outcome must produce an Outcome_Attribution. Claims that a tactic 'works' must be backed by attribution evidence, not assertion.

OUTCOME_ATTRIBUTION — Field Specification

Field	Type	Business Rule	Source
attribution_id	UUID	Immutable	ATTR-RECORD-001
outcome_type	Enum	REPLY / MEETING / PILOT / WIN / LOSS / OPT_OUT	ATTR-RECORD-001
account_id / contact_id	UUID	Subject	—
contributing_touches	Array[Object]	Multi-touch credit weights	ATTR-MULTITOUCH-001
linked_chain/prompt/window/connector	Array[UUID]	Causal contributors	ATTR-RECORD-001
feedback_entry_id	UUID	Link to recalibration	ATTR-FEEDBACK-001

ENTITY 31: TIMING_RECORD

The analytics ledger of timing decisions and their results — when the system acted, why then, and whether the timing paid off.

WHY #1 — Why track timing as its own analytics entity?

Timing is the primary competitive weapon; to sharpen it, the system must measure timing decisions specifically, not bury them in general outcomes.

NEW RULE — TIMING-RECORD-001: Each timing-sensitive decision records when action was taken, the window state at the time, the rationale for that timing, and the eventual result.

WHY #2 — Why capture window-state-at-action?

To learn the optimal moment, the system must correlate results with where in the window it acted (early/peak/closing). Without window state, timing learning is impossible.

NEW RULE — TIMING-WINDOWSTATE-001: Each Timing_Record stores the Buying_Window state at action time. Correlating state with outcomes calibrates the ideal action point within windows.

WHY #3 — Why record missed-timing (acted too early/late)?

Both prematurity and lateness are failures (timing failure cascades). Recording near-misses and misses teaches the system to correct its timing model.

NEW RULE — TIMING-MISS-001: Acted-too-early and acted-too-late outcomes are explicitly recorded, including windows missed entirely. These feed window-prediction and urgency models to reduce future mistiming.

WHY #4 — Why feed timing analytics back into window prediction?

Timing learning must improve future forecasts. Records should recalibrate decay curves and closure predictions.

NEW RULE — TIMING-CALIBRATE-001: Timing_Records recalibrate signal decay curves and Buying_Window closure predictions. Persistent mistiming on a signal type adjusts that type's decay function.

WHY #5 — Deepest reason this entity exists?

It is how the system turns timing from intuition into a measured, improving discipline — operationalizing 'timing as competitive weapon.' It exists because the right message at the wrong time still loses.

NEW RULE — TIMING-PRIMACY-001: Timing decisions of material consequence must produce a `Timing_Record`. Timing models may only be adjusted on the basis of recorded timing evidence.

TIMING_RECORD — Field Specification

Field	Type	Business Rule	Source
timing_record_id	UUID	Immutable	TIMING-RECORD-001
account_id / decision_ref	UUID	Subject decision	—
action_time / window_state_at_action	Timestamp/Enum	When + where in window	TIMING-WINDOW-STATE-001
timing_rationale	Text	Why then	TIMING-RECORD-001
timing_outcome	Enum	OPTIMAL / TOO_EARLY / TOO_LATE / MISSED	TIMING-MISS-001
calibration_impact	Object	Feeds decay/closure models	TIMING-CALIBRATION-001

ENTITY 32: FEEDBACK_LOOP_ENTRY

A single learning event: an observation that adjusts a weight, threshold, pattern, or model — the unit of system self-improvement.

WHY #1 — Why model learning as discrete entries?

Improvement must be auditable and reversible. Discrete feedback entries make each adjustment traceable to its evidence, rather than weights drifting invisibly.

NEW RULE — FEEDBACK-RECORD-001: Each system adjustment (weight change, threshold shift, pattern update, prompt promotion) is recorded as a `Feedback_Loop_Entry` citing the evidence that justified it.

WHY #2 — Why require evidence linkage for every adjustment?

Unjustified self-modification is dangerous. Requiring evidence prevents the system from changing itself on noise and enables rollback of bad changes.

NEW RULE — FEEDBACK-EVIDENCE-001: No adjustment may occur without a linked evidence basis (`Outcome_Attribution`, `Timing_Record`, review patterns). Adjustments failing significance thresholds are rejected.

WHY #3 — Why guard against feedback-loop instability?

Naive feedback can oscillate or overfit recent data (chasing noise). The system must damp changes and protect immutable principles from erosion.

NEW RULE — FEEDBACK-STABILITY-001: Adjustments are rate-limited and damped to prevent oscillation/overfitting. Feedback may never alter immutable meta-principles; it operates only within governed operational bounds.

WHY #4 — Why make every adjustment reversible?

A learning change can degrade performance. Reversibility lets the system undo harmful self-modifications quickly.

NEW RULE — FEEDBACK-REVERSIBLE-001: Every `Feedback_Loop_Entry` is reversible; a degradation in monitored performance can roll back recent adjustments. Rollbacks are logged as `System_Events`.

WHY #5 — Deepest reason this entity exists?

It is the disciplined heartbeat of a learning system — improvement that is evidence-based, bounded, auditable, and reversible. It exists so the system gets better without ever betraying its principles or chasing noise.

NEW RULE — FEEDBACK-PRIMACY-001: All self-improvement flows through Feedback_Loop_Entries. Silent, unlogged, or unbounded self-modification is prohibited.

FEEDBACK_LOOP_ENTRY — Field Specification

Field	Type	Business Rule	Source
feedback_id	UUID	Immutable	FEEDBACK-RECORD-001
adjustment_type	Enum	WEIGHT / THRESHOLD / PATTERN / PROMPT / MODEL_ROUTING	FEEDBACK-RECORD-001
evidence_links	Array[UUID]	Attribution/Timing/Review basis	FEEDBACK-EVIDENCE-001
change_delta	Object	Before/after values	FEEDBACK-RECORD-001
stability_guard	Object	Rate-limit/damping applied	FEEDBACK-STABILITY-001
reversible_to	UUID	Rollback pointer	FEEDBACK-REVERSIBLE-001

ENTITY 33: PERFORMANCE_BASELINE

The reference benchmark of how the system performs, against which improvements, regressions, and experiments are judged.

WHY #1 — Why maintain explicit baselines?

Improvement is meaningless without a reference. Baselines define 'normal,' making it possible to detect whether a change helped, hurt, or did nothing.

NEW RULE — BASELINE-RECORD-001: The system maintains Performance_Baselines per key metric (response rate, conversion, hallucination rate, timing accuracy, capital efficiency). All experiments are judged against the relevant baseline.

WHY #2 — Why baseline per metric and per segment?

A global baseline hides where the system is weak. Per-segment baselines (by account type, signal type) reveal targeted strengths and gaps.

NEW RULE — BASELINE-SEGMENT-001: Baselines are maintained per metric AND per segment. Regressions in any segment are detectable even when the global average looks healthy.

WHY #3 — Why require regression detection against baselines?

Silent regressions erode performance over time. Active comparison to baseline catches degradation early — including from self-improvement gone wrong.

NEW RULE — BASELINE-REGRESSION-001: Monitored metrics are continuously compared to baseline. Significant regression triggers alerts and can auto-pause the responsible change (feedback rollback, prompt/model rollback).

WHY #4 — Why re-baseline deliberately rather than continuously?

If the baseline silently tracks current performance, you can't detect drift. Baselines must be updated deliberately and recorded, preserving the ability to see long-term change.

NEW RULE — BASELINE-REBASE-001: Re-baselining is an explicit, logged governance action, not automatic. Prior baselines are retained so long-term trajectory (improvement or decay) remains visible.

WHY #5 — Deepest reason this entity exists?

It is the system's standard of truth about its own performance — the guardrail that distinguishes real progress from illusion and catches decay before it compounds. It exists so the system's self-assessment stays honest.

NEW RULE — BASELINE-PRIMACY-001: Claims of improvement require beating the recorded baseline with significance. The baseline is the authoritative reference for all performance judgments and experiment evaluation.

PERFORMANCE_BASELINE — Field Specification

Field	Type	Business Rule	Source
baseline_id	UUID	Immutable version node	BASELINE-RECORD-001
metric / segment	Enum/Object	What + where measured	BASELINE-SEGMENT-001
baseline_value / confidence_interval	Object	Reference performance	BASELINE-RECORD-001
regression_thresholds	Object	Alert/auto-pause triggers	BASELINE-REGRESSION-001
established_at / superseded_at	Timestamp	Re-baseline history	BASELINE-REBASE-001

SECTION 2 — COMPLETION SUMMARY

All 33 entities across 5 tiers have been excavated via full 5-Whys, mirroring the Section 1 methodology. Each entity now carries an explicit definition, a five-level causal justification, surfaced data-layer rules, and a complete field specification ready for engineering handoff.

Tier 1 — Core	5 entities	25 Whys	~23 rules
Tier 2 — Intelligence	10 entities	50 Whys	~48 rules
Tier 3 — Decision	8 entities	40 Whys	~40 rules
Tier 4 — Governance	6 entities	30 Whys	~30 rules
Tier 5 — Analytics	4 entities	20 Whys	~20 rules
TOTAL	33 entities	165 Whys	163 rules

Next: Section 3 — Modules & Workflows, building on this entity model. The entities defined here are the nouns; the modules and workflows are the verbs that operate on them.